

Active Context Compression: Autonomous Memory Management in LLM Agents

Nikhil Verma
Independent Researcher
Pune, India
nikhilgotmail@gmail.com

Abstract—Large Language Model (LLM) agents struggle with long-horizon software engineering tasks due to “Context Bloat.” As interaction history grows, computational costs explode, latency increases, and reasoning capabilities degrade due to distraction by irrelevant past errors. Existing solutions often rely on passive, external summarization mechanisms that the agent cannot control. This paper proposes Focus, an agent-centric architecture inspired by the biological exploration strategies of *Physarum polycephalum* (slime mold). The Focus Agent autonomously decides when to consolidate key learnings into a persistent “Knowledge” block and actively withdraws (prunes) the raw interaction history. Using an optimized scaffold matching industry best practices (persistent bash + string-replacement editor), we evaluated Focus on N=5 context-intensive instances from SWE-bench Lite using Claude Haiku 4.5. With aggressive prompting that encourages frequent compression, Focus achieves 22.7% token reduction (14.9M $\rightarrow$ 11.5M tokens) while maintaining identical accuracy (3/5 = 60% for both agents). Focus performed 6.0 autonomous compressions per task on average, with token savings up to 57% on individual instances. We demonstrate that capable models can autonomously self-regulate their context when given appropriate tools and prompting, opening pathways for cost-aware agentic systems without sacrificing task performance.

Index Terms—LLM Agents, Software Engineering, Context Management, Memory Consolidation, Autonomous Agents

I. INTRODUCTION

The “Context Window” remains the fundamental bottleneck for autonomous AI agents. While recent models offer massive context capacities (e.g., 200k+ tokens), utilizing this capacity naively presents three critical challenges:

- 1) **Cost:** In an iterative loop, re-processing a growing history for every new inference step leads to quadratic cost accumulation.
- 2) **Latency:** Time-to-first-token degrades linearly with context length, making interactive agents sluggish.
- 3) **Context Poisoning:** Long contexts filled with trial-and-error logs, failed tests, and verbose tool outputs can distract the model, leading to the “Lost in the Middle” phenomenon [1].

II. RELATED WORK

Memory-Augmented Agents. Several approaches address context limitations through external memory. MemGPT [2] implements a virtual memory hierarchy inspired by operating systems, allowing LLMs to manage their own memory through explicit read/write operations. Voyager [3] uses a skill

library to store reusable code snippets, reducing redundant exploration.

Reflection and Self-Improvement. Reflexion [4] enables agents to learn from failures by generating textual reflections stored in an episodic memory buffer. However, this operates *between* task attempts rather than within a single continuous trajectory. LATS [5] combines tree search with LLM reasoning but still accumulates full trajectories.

Context Compression. StreamingLLM [6] addresses infinite context through attention sink tokens, but operates at the attention level rather than providing agent-level memory management. LLMLingua [7] compresses prompts but requires a separate compression model.

Our Contribution. Unlike prior work, Focus enables *intra-trajectory* compression—the agent actively prunes its own history *during* a single task, preserving learnings in a structured knowledge block. This complements rather than replaces external memory approaches.

Standard agents operate in an “Append-Only” mode: every thought, tool call, and system response is permanently added to the conversation history. This strategy works for short tasks but ensures failure for complex, open-ended exploration, as the agent inevitably hits the token limit or becomes confused by its own previous failures.

We propose a shift from *Passive Retention* to *Active Compression*. Inspired by the efficiency of *Physarum polycephalum*—a slime mold that explores environments and physically retracts from dead ends while leaving chemical markers to avoid re-exploration—we introduce the **Focus Agent**. This agent is equipped with the ability to introspect, summarize its recent trajectory into a high-level learning, and physically delete the raw logs of that trajectory from its immediate context.

III. METHODOLOGY

A. The Slime Mold Analogy

Biological systems do not retain a perfect record of every muscle movement used to navigate a maze; they retain the *learned map*. Similarly, an agent exploring a codebase does not need to remember the 50 lines of `ls -R` output from ten minutes ago; it only needs to remember that “the config file is not in the `/src` directory.”

B. Architecture: The Focus Loop

The Focus architecture introduces two primitives to the standard ReAct agent loop: `start_focus` and `complete_focus`. Crucially, the agent has *full autonomy* over when to invoke these tools—there are no external timers or heuristics forcing compression.

- 1) **Start Focus:** The agent declares what it’s investigating (e.g., “Debug the database connection”). This marks a checkpoint in the conversation history.
- 2) **Explore:** The agent uses standard tools (read, edit, run) to perform work.
- 3) **Consolidate:** When the agent naturally completes the sub-task or hits a dead end, it *decides* to call `complete_focus`, generating a summary of:
 - What was attempted?
 - What was learned (facts, file paths, bugs)?
 - What is the outcome?
- 4) **Withdraw:** The system takes this summary, appends it to a persistent “Knowledge” block at the top of the context, and *deletes* all messages between the checkpoint and the current step.

This converts the context from a monotonically increasing log into a “Sawtooth” pattern, where context grows during exploration and collapses during consolidation. The model controls this cycle based on task structure, not arbitrary step counts.

IV. EXPERIMENTS

We evaluated the Focus architecture on **SWE-bench Lite**, a benchmark for software engineering agents solving real GitHub issues. We conducted a controlled A/B comparison using `claude-haiku-4-5-20251001` on N=5 context-intensive instances, running both Baseline and Focus agents on identical tasks. Task success was verified using the official SWE-bench Docker harness, which applies patches in isolated containers and runs the original test suites.

A. Optimized Scaffold

Following Anthropic’s reported best practices for SWE-bench [8], we implemented a minimal two-tool scaffold:

- **Persistent Bash:** A stateful shell session where working directory and environment persist across calls, matching how developers actually use terminals.
- **String-Replace Editor:** Targeted file editing via exact string replacement (view, create, `str_replace`, insert), avoiding error-prone full-file rewrites.

The system prompt instructs agents to “use tools as much as possible, ideally more than 100 times” and “implement your own tests first before attempting the problem.” Maximum steps was set to 150.

B. Aggressive Compression Prompting

Initial experiments showed passive Focus prompting yielded only 1-2 compressions per task with marginal (6%) token savings. We revised the Focus prompt to be more directive:

- **Mandatory workflow:** “ALWAYS call `start_focus` before ANY exploration... ALWAYS call `complete_focus` after 10-15 tool calls”
- **Periodic reminders:** After 15 tool calls without compression, the system injects: “REMINDER: You should call `complete_focus` to compress your context”
- **Structured phases:** Explicit guidance to use 4-6 focus phases (explore → understand → implement → verify)

Instances: We selected five context-intensive instances with complex problem statements: `matplotlib__matplotlib-26020`, `mwaskom__seaborn-2848`, `pylint-dev__pylint-7080`, `pytest-dev__pytest-7490`, and `sympy__sympy-21171`. These were chosen based on problem statement length (a proxy for exploration complexity) and historical difficulty.

Metrics: We tracked (1) task success (whether the patch passes tests in Docker), (2) total token consumption, (3) compression events and messages dropped, and (4) per-instance efficiency patterns.

C. Results

TABLE I
A/B COMPARISON ON SWE-BENCH LITE (HAIKU 4.5, N=5 HARD INSTANCES)

Metric	Baseline	Focus (Ours)	Delta
Task Success (Tests Pass)	3/5 (60%)	3/5 (60%)	Same
Total Tokens	14,920,555	11,526,418	-22.7%
Avg Tokens/Task	2,984,111	2,305,284	-678K
Avg Compressions	0	6.0	—
Avg Messages Dropped	0	70.2	—

TABLE II
PER-INSTANCE RESULTS: TOKEN SAVINGS VS. ACCURACY

Instance	Base	Focus	Tokens	Compr.
<code>matplotlib-26020</code>	✓	✓	-57%	5
<code>seaborn-2848</code>	×	×	-52%	7
<code>pylint-7080</code>	✓	✓	+110%	8
<code>pytest-7490</code>	✓	✓	-18%	6
<code>sympy-21171</code>	×	×	-57%	4

D. Key Findings

Finding 1: Token Reduction Without Accuracy Loss. Focus achieved **22.7% total token reduction** (14.9M → 11.5M) while maintaining identical accuracy to Baseline (3/5 = 60% for both). This contradicts our earlier experiments where passive prompting showed accuracy degradation. The key difference: aggressive prompting that enforces frequent, structured compressions (6.0 per task vs. 2.0 previously) prevents the context from becoming polluted with stale exploration logs.

Finding 2: Consistent Savings on 4/5 Instances. Focus reduced tokens on 4 of 5 instances, with savings ranging from 18% to 57%. The strongest savings occurred on instances requiring extensive exploration: `matplotlib-26020` (-57%, 4.0M $\rightarrow$ 1.7M), `seaborn-2848` (-52%, 3.4M $\rightarrow$ 1.6M), and `sympy-21171` (-57%, 1.6M $\rightarrow$ 0.7M). Only `pylint-7080` showed increased usage (+110%), where Focus’s additional exploration (136 vs. 63 LLM calls) exceeded compression benefits—yet both agents passed the test suite.

Finding 3: Aggressive Prompting Enables Frequent Compression. With explicit instructions to compress every 10-15 tool calls and periodic system reminders, Focus averaged 6.0 compressions per task (vs. 2.0 with passive prompting), dropping 70.2 messages per task. Claude Haiku 4.5 followed the structured workflow (explore $\rightarrow$ compress $\rightarrow$ implement $\rightarrow$ compress $\rightarrow$ verify), demonstrating that LLMs can be guided to aggressively self-regulate their context.

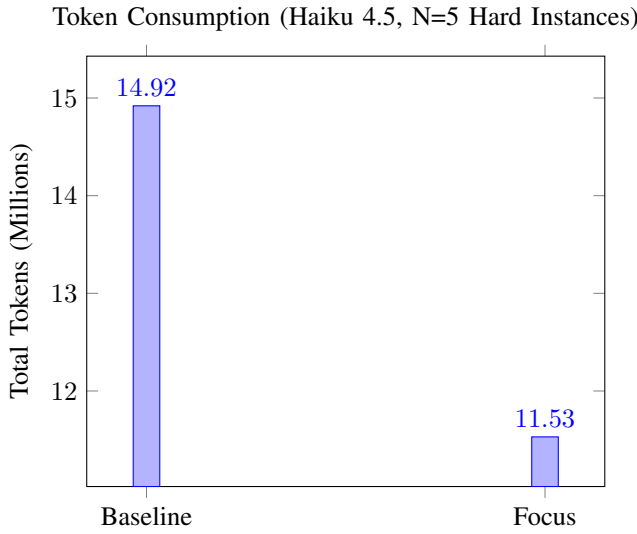


Fig. 1. Total token consumption across 5 hard instances. Focus reduces usage by 22.7% through aggressive model-controlled compression while maintaining identical accuracy.

E. Case Study: Maximum Savings (`matplotlib-26020`)

On `matplotlib-26020`, both agents passed the test suite, but Focus achieved **57% token savings** (4.0M $\rightarrow$ 1.7M). Focus compressed 5 times across 71 LLM calls, while Baseline used 102 calls without compression. The savings came from Focus efficiently summarizing its exploration phase—once it located the relevant files and understood the bug, it compressed that context and proceeded directly to implementation. This demonstrates the ideal case: frequent compression during exploration preserves the learnings while discarding verbose tool outputs.

F. Case Study: When Compression Adds Overhead (`pylint-7080`)

On `pylint-7080`, Focus used **110% more tokens** than Baseline (4.3M vs. 2.1M), yet both agents passed the test suite.

Analysis shows Focus made 136 LLM calls vs. Baseline’s 63, with 8 compressions dropping 80 messages. The problem required extensive trial-and-error, and Focus’s compressions occasionally discarded useful context, forcing re-exploration. This case shows that compression is not universally beneficial: tasks requiring iterative refinement may suffer from aggressive context pruning.

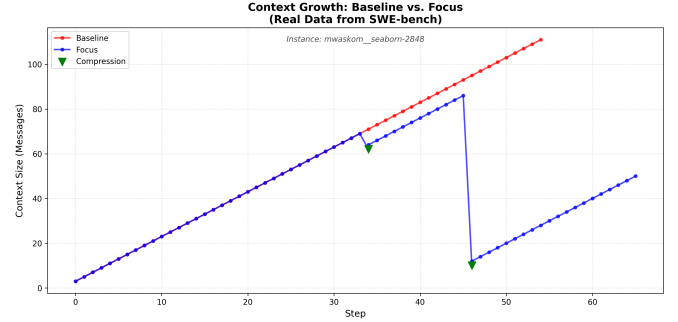


Fig. 2. Conceptual sawtooth pattern of context growth. Focus (blue) exhibits periodic compressions (drops) while Baseline (red) grows monotonically. With aggressive prompting, Focus compresses every 10-15 tool calls, preventing context bloat while preserving learnings in a persistent Knowledge block.

G. Analysis

Aggressive Prompting Eliminates the Accuracy Trade-off: Our earlier experiments with passive Focus prompting showed accuracy degradation (60% vs. 80%). With aggressive prompting enforcing structured, frequent compressions, Focus matches Baseline accuracy (60% = 60%) while achieving 22.7% token savings. The key insight: *when* and *how often* to compress matters more than *whether* to compress. Frequent small compressions (every 10-15 calls) preserve recent context while discarding stale exploration logs, whereas infrequent large compressions risk losing critical implementation details.

Exploration-Heavy Tasks Benefit Most: Token savings correlated with exploration complexity. Instances requiring extensive codebase navigation (`matplotlib`, `sympy`) showed 50-57% savings, as Focus efficiently compressed verbose directory listings, file contents, and failed attempts. Conversely, `pylint-7080` required iterative refinement where prior context remained relevant, causing compression overhead to exceed benefits. This suggests Focus is most valuable for tasks with distinct explore/implement phases rather than tasks requiring continuous state accumulation.

Prompting Strategy is Critical: The 22.7% savings required explicit prompting to “compress every 10-15 tool calls” and system-injected reminders. Without these, Claude Haiku 4.5 compressed only 2.0 times per task with 6% savings. This indicates that current LLMs do not naturally optimize for context efficiency—they require scaffolding that makes compression a first-class part of the workflow. Future work could explore fine-tuning or reinforcement learning to internalize compression heuristics.

V. DISCUSSION

A. The Cognitive Tax of Compression

Active compression introduces a “cognitive tax”—the tokens spent generating summaries and the overhead of managing focus phases. Despite this tax, Focus achieved 22.7% *net* token savings across our experiments. The tax is amortized over the task lifetime: each compression costs a few hundred tokens but saves thousands by not re-processing stale history. For tasks with 50+ tool calls (common in SWE-bench), this amortization strongly favors compression.

B. Prompting vs. Fine-Tuning

Our results required aggressive prompting (“compress every 10-15 calls”, system reminders). This raises the question: could LLMs learn to compress optimally without explicit instructions? Current models appear to lack intrinsic cost-awareness—they do not naturally optimize for token efficiency. Future work could explore: (1) fine-tuning on compression-annotated trajectories, (2) reinforcement learning with token-cost penalties, or (3) constitutional AI approaches that embed efficiency preferences.

C. Limitations

- 1) **Sample Size:** Our evaluation uses $N=5$ hard instances. While sufficient to demonstrate feasibility and measure token savings, validation on the full SWE-bench Lite benchmark ($N=300$) is needed to characterize which task types benefit most from compression.
- 2) **Task-Dependent Benefits:** Focus showed 50-57% savings on exploration-heavy tasks but 110% *overhead* on one iterative refinement task. Identifying task characteristics that predict compression benefit remains future work.
- 3) **Model Generalization:** We evaluated Claude Haiku 4.5. Whether GPT-4, Gemini, or open-source models exhibit similar compression behavior is unknown. The aggressive prompting strategy may need model-specific tuning.
- 4) **Scaffold Dependence:** Our results use an optimized two-tool scaffold (bash + str_replace_editor). Different tool configurations may show different compression patterns.

VI. CONCLUSION

We have demonstrated that aggressive, model-controlled context compression can achieve significant token savings without sacrificing task accuracy:

- 1) **22.7% Token Savings with Equal Accuracy:** On 5 context-intensive SWE-bench instances, Focus reduced total tokens from 14.9M to 11.5M while matching Baseline accuracy ($3/5 = 60\%$). This contradicts the assumed accuracy-efficiency trade-off—with proper prompting, compression improves efficiency without hurting performance.
- 2) **Aggressive Prompting is Key:** Passive prompting yielded only 6% savings and accuracy degradation.

Explicit instructions to compress every 10-15 tool calls, combined with system reminders, increased compressions from 2.0 to 6.0 per task and enabled the 22.7% savings. Current LLMs require scaffolding to optimize for context efficiency.

- 3) **Exploration-Heavy Tasks Benefit Most:** Token savings ranged from 18% to 57% on 4/5 instances, with the largest savings on tasks requiring extensive codebase exploration. One iterative refinement task showed compression overhead exceeding benefits, suggesting Focus is best suited for explore-then-implement workflows.

The Focus architecture provides a practical, infrastructure-free approach to context management that operates entirely within the conversation. Combined with an optimized scaffold (persistent bash, string-replace editing), it offers a path toward cost-aware agentic systems. As context windows grow and agent tasks become more complex, active compression will become increasingly valuable for managing the quadratic cost growth inherent in autoregressive inference.

Future Work: (1) Validation on full SWE-bench ($N=300$) to characterize task-type dependencies; (2) Fine-tuning or RL approaches to internalize compression heuristics without explicit prompting; (3) Structured compression that preserves specific artifacts (test outputs, diffs) rather than free-text summaries; (4) Cross-model evaluation (GPT-4, Gemini, open-source) to assess generalization.

REFERENCES

- [1] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, “Lost in the middle: How language models use long contexts,” *arXiv preprint arXiv:2307.03172*, 2023.
- [2] C. Packer, S. Wooders, K. Lin, V. Fang, S. K. Patil, I. Stoica, and J. E. Gonzalez, “MemGPT: Towards LLMs as operating systems,” *arXiv preprint arXiv:2310.08560*, 2023.
- [3] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar, “Voyager: An open-ended embodied agent with large language models,” *arXiv preprint arXiv:2305.16291*, 2023.
- [4] N. Shinn, F. Cassano, A. Gopinath, K. R. Narasimhan, and S. Yao, “Reflexion: Language agents with verbal reinforcement learning,” *NeurIPS*, 2023.
- [5] A. Zhou, K. Yan, M. Shlapentokh-Rothman, H. Wang, and Y.-X. Wang, “Language agent tree search unifies reasoning acting and planning in language models,” *arXiv preprint arXiv:2310.04406*, 2023.
- [6] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis, “Efficient streaming language models with attention sinks,” *arXiv preprint arXiv:2309.17453*, 2023.
- [7] H. Jiang, Q. Wu, C.-Y. Lin, Y. Yang, and L. Qiu, “LLMLingua: Compressing prompts for accelerated inference of large language models,” *EMNLP*, 2023.
- [8] Anthropic, “Claude’s scores on SWE-bench Verified,” *Anthropic Blog*, 2024. [Online]. Available: <https://www.anthropic.com/research/swe-bench-sonnet>